

AXONA

Machine-Oriented Grounding for AI-Built Apps

David A. Smith

Axona.net

The Axona AI Grounding v4.22.0 2026-07-14

Axona AI Grounding — kernel 4.22.0

This file is the complete, self-contained grounding for an AI system building an application on the Axona protocol. It matches the network it targets: **kernel 4.22.0 / wire 4.0, deployed on testnet (wss://testnet.axona.net)**. Everything below is exact and current; nothing outside this file is required. If this version does not match the bridge you are connecting to, request the matching grounding file.

Axona is a serverless peer-to-peer pub/sub network. Every participant is a peer. Topics are addresses derived from descriptors; messages route to an emergent per-topic root and fan out through a distribution tree. There is no central server, message broker, or database.

HARD RULES — violations cause silent failure or thrown errors

1. **A topic is a descriptor object, never a string.** { region?, owner?, name, write? }. Publisher and subscriber must use IDENTICAL fields — the topic ID is a pure hash of them. A mismatch (one names region, the other omits it) means two different topics and **no delivery, with no error**.
2. **Every publish names its signer.** peer.pub(topic, msg, { signWith }) — signWith is an author identity or the ANONYMOUS sentinel. Omitting it throws PUBLISH_NO_PUBLISH_IDENTITY. There is no default signer.
3. **Two identities, never interchangeable.** The node identity (createNodeIdentity) is the connection — it has a location and forms the node ID; it NEVER signs content. The author identity (createAuthorIdentity) signs content — it has NO location and NO node ID.
4. **await peer.ready() before first pub/sub.** Publishing on a cold mesh is survivable (the kernel retries) but unpredictable. ready() resolves when the routing mesh is warm.
5. **A publish returns no delivery acknowledgment — ever.** This is a privacy invariant (author and transport identities must stay uncorrelated). To confirm delivery, subscribe to the topic yourself and watch for your own msgId to arrive.
6. **Message payloads: 15 KB is the reliable cross-browser floor.** Hard ceiling 256 KB, but anything over 15 KB may fail on real WebRTC channels. For larger payloads use std/chunk (below), never a bigger single publish.
7. **Messages expire.** Default hold ~24 h (48 h ceiling). Re-publishing the identical payload from the same author yields the SAME msgId and refreshes the hold (an upsert; subscribers are not re-notified). Axona is a live messaging fabric, not permanent storage.
8. **kill is the only retraction,** must be signed by the same author key that signed the original, and is best-effort (receivers already holding the plaintext keep it). Anonymous messages cannot be killed. unpub and touch do not exist in 4.x — do not call them.
9. **Persist the author key if user identity should survive restarts.** createAuthorIdentity({ persistAs: 'myapp:author' }) (browser localStorage). Everything else (node identity, subscriptions) can be ephemeral and re-created each run.
10. **Node 20+ or a browser over HTTPS.** The kernel needs Web Crypto (crypto.subtle) and ES modules. No build step, no bundler required.

Install

npm install github:axona-net/axona-protocol#v4.22.0

package.json must contain "type": "module".

```
import { connect } from '@axona/protocol/connect.js'; // the one-call bootstrap
import {
  AxonaPeer, AxonaDomain, NeuronNode, ANONYMOUS,
  createNodeIdentity, createAuthorIdentity,
  deriveTopicId, metricTopic, KERNEL_VERSION,
} from '@axona/protocol';
import { webTransport } from '@axona/protocol/transport/web/index.js';
import { makeMessage, readMessage } from '@axona/protocol/std/message.js';
import { publishChunkedBytes, receiveChunkedBytes } from '@axona/protocol/std/chunk.js';
```

Canonical peer bootstrap (copy exactly)

```
const { peer, author, status, disconnect } = await connect({
  bridge: 'wss://testnet.axona.net',
  location: { lat: 38.0, lng: -77.0 }, // the user's real location
  author: 'myapp:author', // string = durable (persistAs); true = ephemeral;
  ↪ false = none
});
// status: { ready, peers, ms, reason } - never rejects; ready:false on timeout

Teardown: await disconnect();
```

Options: k (routing set size, default 20), ready (forwarded to peer.ready(); false skips the wait), transport/nodeIdentity (injection for tests or custom stacks), web (extra webTransport options).

Alternative: manual assembly (custom transports / multiple peers)

```
const nodeIdentity = await createNodeIdentity({ lat: 38.0, lng: -77.0 });
const author = await createAuthorIdentity({ persistAs: 'myapp:author' });
const transport = webTransport({ bridgeUrl: 'wss://testnet.axona.net', identity: nodeIdentity
  ↪ });
const node = new NeuronNode({ id: nodeIdentity.id, lat: 38.0, lng: -77.0 }); // hex id
  ↪ accepted (4.14+)
node.transport = transport;
const peer = new AxonaPeer({ domain: new AxonaDomain({ k: 20 }),
  node, nodeIdentity, transport });
await transport.start(nodeIdentity.id);
await peer.start();
const status = await peer.ready();
```

Topics

A topic descriptor: { region?, owner?, name, write? }

Field	Type	Meaning
region	'useast' '0x89' 137	Geographic cell the topic lives in. All three forms are equivalent. Omitted → defaults to the publisher's own region (co-located peers converge; cross-region readers must then know that region). Prefer explicit.
owner	64-hex Author ID absent	Namespaces the topic under an author.
name	non-empty string	The human label. Required.
write	'open' 'owner'	Who may publish. Defaults: no owner → 'open' ; owner present → 'owner' .

The three canonical shapes:

```
const lobby = { region: 'useast', name: 'lobby' }; // open room: anyone
↳ reads/writes
const feed = { region: 'useast', owner: me.authorId, name: 'posts' }; // owner-only feed
↳ (write defaults 'owner')
const inbox = { region: 'useast', owner: me.authorId, name: 'inbox', write: 'open' }; //
↳ anyone writes TO me
```

- `await deriveTopicId(descriptor)` → 66-hex topic ID (a shareable read handle). `sub/pull/metrics` accept a descriptor **or** this ID; `pub` always needs the full descriptor.
- Publishing to someone else's `write:'owner'` topic throws / is rejected at the network with `WRITE_POLICY_VIOLATION`.

Publish / subscribe

```
// Publish (signed). Returns the msgId (64-hex content hash). NO delivery ack.
const msgId = await peer.pub(topic, message, { signWith: author });
// message: any JSON-serializable value. Anonymous: { signWith: ANONYMOUS }.

// Subscribe. handler receives an envelope for EVERY delivery, exactly once per msgId.
const sub = await peer.sub(topic, (env) => {
  if (env.deleted) { /* a retraction: drop env.msgId locally */ return; }
});
```

```

    // env = { msgId, seq, ts, topic, message, signerPubkey?, signature? }
    // env.signerPubkey is the author's ID (absent when anonymous)
  }, { since: 'all' });

// since options: omit = live only; 'latest' = newest cached + live;
//                  'all' = full cached history + live; <unix-ms number> = newer than.

await sub.stop();           // stop this handle
await peer.unsub(topic);    // stop ALL local subs for the topic + network unsubscribe

// Retract (author-only, best-effort):
await peer.kill(topic, msgId, { signWith: author });
// Subscribers receive { deleted: true, msgId, topic } on their normal handler.

```

Delivery semantics an app can rely on: - Exactly-once per `msgId` to each subscriber handler. - Publishing again with the identical payload + author = same `msgId` = refresh, not a duplicate delivery. - Late subscribers with `since: 'all'` receive cached history (bounded queue, ~24 h) then the live tail. - The kernel retries a publish internally until it confirms placement; a freshly-joined publisher gets an extra rapid burst. Your code does not retry.

Reads without subscribing

```

const env = await peer.pull(msgId, { topic });           // exact message by content hash, or
↳ null
const last = await peer.pull(null, { topic });           // topic's newest (may be slightly
↳ stale – served by the nearest replica ON PURPOSE)

```

If the strict newest matters, subscribe instead of polling `pull(null)`. A successful pull extends the message's hold a little.

Topic activity (metrics)

```

const m = await peer.metrics(topic); // one-shot: latest published snapshot
// Live: subscribe to the derived metric topic
const mt = metricTopic(await deriveTopicId(topic)); // -> a topic descriptor
await peer.sub(mt, (env) => {
  const snap = JSON.parse(env.message); // { topic, ts, subscribers, current_count, seq, ...
  ↳ }
}, { since: 'latest' });

```

TIMING RULES (metrics are published ON DEMAND — the subscribe itself turns them on; there is no always-on publisher):

- First snapshot: at ROUTING LATENCY (~0.3 s typical; allow a few seconds under churn) — the root answers the moment the lease arms, independent of other users' publish/watch activity. Cadence: every ~20 s while subscribed. Publishing stops ~70 s after the last metric subscriber leaves.

- `peer.metrics()` (one-shot) normally succeeds within its 1.5 s default window; `stale: true` means the answer didn't arrive in time (churn) — retry or prefer the standing `sub()` form.
- Render silence as UNKNOWN, never as zero: a `current_count: 0` snapshot is the real “no activity” answer. Keep the subscription open in tests.

Metrics are advisory (unauthenticated counts) — never a security input.

Direct messages (peer-to-peer, not pub/sub)

```
const reply = await peer.send(targetNodeIdHex, { any: 'json' }); // RPC: awaits remote
↳ handler's return
peer.notify(targetNodeIdHex, { any: 'json' }); // fire-and-forget
peer.onMessage(async (fromNodeIdHex, message) => { return { ok: true }; });
```

Target must be a currently-known peer (`peer.peers()` lists node IDs). Direct messages address a *node* (device/session), not an author.

Presence / introspection

```
peer.peers() // -> ['<66-hex nodeId>', ...] currently in the routing table
peer.onPeerJoin(id => {}); peer.onPeerLeave(id => {});
peer.health() // { nodeId, synaptomeSize, subscriptions, hosting, wireVersion,
↳ ... }
peer.onLog('warn', (evt, data) => {}); peer.onError(err => {});
peer.onUpgradeRequired(() => { /* show "please update" */ });
```

Large payloads (files, images) — std/chunk

```
// Sender: splits into signed sub-15KB chunks, publishes each.
await publishChunkedBytes(peer, bytes /* Uint8Array */, {
  topic, signWith: author,
  meta: { filename: 'photo.jpg', mime: 'image/jpeg' },
});
// Receiver: reassembles complete payloads.
await receiveChunkedBytes(peer, topic, {
  onComplete: ({ bytes, meta }) => { /* whole file */ },
  onProgress: ({ received, total }) => {},
});
```

Interoperable message bodies — std/message

If other Axona apps may read your topic, wrap text bodies:

```
await peer.pub(topic, makeMessage('hello', { customField: 1 }), { signWith: author });
const { text } = readMessage(env.message); // tolerant reader
```

Serving without consuming — host

```
await peer.host();           // host this node's keyspace neighborhood (relay mode)
await peer.host(topic);      // host one topic: store + serve it without subscribing
await peer.unhost(topic);
```

Use in long-running Node processes to keep topic history alive; browser apps rarely need it.

Agent legibility (recommended for AI-operated authors)

```
await peer.setAuthorClass('agent', { signWith: author, label: 'my-bot' });
const cls = await peer.getAuthorClass(someAuthorId); // { class:'agent' | 'human' | 'unstated',
↪ ... }
```

An AI publishing as an autonomous agent SHOULD declare 'agent'. It is voluntary provenance; absence reads as 'unstated', never 'human'.

Envelope shape (what handlers receive)

```
{
  msgId:      "<64-hex sha256(canonical({publisher, message}))>",
  seq:        1716638400123,           // per-author monotonic
  ts:         1716638400000,           // author's clock (ms)
  topic:      { region, owner, name, write }, // the SIGNED descriptor
  message:    <your payload>,
  signerPubkey: "<64-hex author id>", // absent if anonymous
  signature:   "ed25519:<128-hex>"    // absent if anonymous
}
```

Signatures are verified by the kernel at network ingress AND on delivery; `env.signerPubkey` present means verified. The signature binds the whole descriptor, so a message cannot be replayed onto a different topic.

Limits

Limit	Value
Publish size (hard)	256 KB
Publish size (reliable floor — use std/chunk above this)	15 KB
Message hold	~24 h default, 48 h ceiling; re-publish refreshes
Per-topic cached history	bounded queue (late joiners see recent history, not everything ever)
Per-publisher share of an open topic's queue	quota-bounded (one author cannot flood)
Live-publish freshness window	signed <code>ts</code> within ± 5 min of the root's clock

Error codes (err.code on thrown AxonaError)

Code	Thrown by	Meaning / fix
PUBLISH_NO_PUBLISH_IDENTITY	pub	Missing <code>signWith</code> . Pass an author or <code>ANONYMOUS</code> .
WRITE_POLICY_VIOLATION	pub	Signer $\neq$ owner on a <code>write:'owner'</code> topic.
TOPIC_REGION_REQUIRED	topic derivation	Open topic with no derivable region. Name a <code>region</code> .
PUBLISH_PAYLOAD_TOO_LARGE	pub	Over 256 KB. Chunk it.
PUBLISH_INVALID_TOPIC / SUBSCRIBE_INVALID_TOPIC	pub/sub	Malformed descriptor (e.g. empty <code>name</code>).
KILL_SIGN_FAILED	kill	<code>signWith</code> is not the original author key.
UPGRADE_REQUIRED (WS close 4426)	connect	Kernel/wire mismatch with the bridge. Install the tag matching this file.
TRANSPORT_PEER_UNREACHABLE	send	Target node not connected. Check <code>peer.peers()</code> .

`pull` returning `null` and `kill` resolving `{ ok:false }` are normal outcomes (missing/expired message; nothing to retract) — not errors.

Troubleshooting rules

- **Connected but nothing arrives** → publisher and subscriber derived different topic IDs. Diff the descriptors field-by-field (including an omitted vs named `region`), or → published before `ready()`.
- **Cannot mix BigInt and other types** → pre-4.14 kernel with a hex id passed to `NeuronNode`; on this version the hex id is accepted directly (and `connect()` avoids the question entirely).
- **Works on localhost, fails deployed** → page not HTTPS (`crypto.subtle` unavailable).
- **Large message silently missing for some receivers** → over the 15 KB reliable floor. Use `std/chunk`.
- **User identity resets every reload** → author not persisted. Use `createAuthorIdentity({ persistAs })`.
- **WS close 4426** → version mismatch; reinstall the pinned tag.

Environment notes

- Browser: HTTPS only. Node: v20+; the same `webTransport` connects to the bridge over WSS (the WebRTC mesh is browser-side; Node peers converse via the bridge and routing).
- The testnet bridge is `wss://testnet.axona.net` (kernel 4.22.0, wire 4.0) — the network this grounding targets. Production (`wss://bridge.axona.net`) runs the same wire-4 line, typically

one release behind; the two are wire-compatible but SEPARATE networks (a peer joins one or the other).

- Multiple tabs = independent peers (fine, but each is a separate node).

End of grounding. Human-oriented companions: the Axona Quick Start, Programmer Guide, and API Reference (same version).